Supplementary analytical scripts for Noctuidae mitochondrial genome evolution

Clara I. Saldamando Benjumea | Manuscript GDNA FLRP 2026 0036 R2 | Reproducibility review package

This document supplies the same portable scripts included in the accompanying repository package, their execution instructions and the recovered historical code. Reconstructed implementations are identified explicitly. Python table and figure generation was executed against the preserved inputs; external phylogenetic and codon model programs were not rerun.

Data and execution

The input manifest contains 30 noctuid accessions. The original 40 GenBank records include nine excluded identical records and the Ostrinia nubilalis outgroup MN793322.1. Annotated CDS are extracted in their annotated orientation, codon_start is applied and NCBI genetic code 5 defines sense codons. All 390 retained CDS have codon_start 1. Stop codons, ambiguous triplets and incomplete terminal triplets are excluded.

Run the commands below from the extracted repository directory in Ubuntu. Portable scripts determine their input paths relative to their own location and write into results. Archived data and reference files are not modified.

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python scripts/01_reproduce_tables.py
python scripts/02_prepare_phylogeny.py
python scripts/03_plot_results.py
```

The dependency versions used for the verification are pinned in requirements.txt. Optional program runners require CodonW, IQ-TREE, MrBayes and CODEML to be installed independently on PATH. Their commands and settings are recorded in 04_run_external.py and in the original configuration files.

Validation and limits

Reconstructed RSCU agrees with 1860 archived accession codon values and 868 species codon means within 0.000001. The reconstructed CodonW input matches all 30 archived sequences and contains 111739 sense codons. Nine duplicate sequence pairs were verified. The neutrality fit gives intercept 27.21927049, slope 0.11379699, R squared 0.32014902 and p 0.03494254. Gene category values and correlations agree with the rounded manuscript supplement. The final 31 sequence alignment of 11229 positions is reconstructed exactly from the archived cleaned gene alignments.

Species RSCU means average individual accession values. Gene RSCU pools codons across accessions. Amino acid families absent from a pooled gene have undefined RSCU and are excluded from its mean A or T ending RSCU. Gene GC3s uses positions permitting a synonymous third base substitution. PR2 counts all retained third positions. These metrics must not be conflated with archived CodonW indices.

The original command and detailed cleaning decisions preceding the cleaned MACSE alignments remain incompletely recovered. Consequently the phylogenetic preparation is reproducible from the archived cleaned alignments, rather than a validated reconstruction of the full alignment procedure from unaligned GenBank CDS. The manuscript should reconcile descriptions of R graphics with the recovered Python plotting code. CodonW code numbering requires verification in the original program. External model estimates are imported historical results until their optional reruns are compared.

Portable reconstructed scripts

The following four source files are reconstructed implementations. Their file names and contents are identical to those supplied in scripts. Plot styles are new renderings of the same analyses and may differ from archived figure

artwork. The external runner is provided for future execution and was not run during this verification.

01_reproduce_tables.py

Repository path scripts/01_reproduce_tables.py

```
#!/usr/bin/env python3
"""Reconstructed reproducibility implementation, not recovered original code.

Run from any directory. Read preserved GenBank records and accession manifest;
write new results without modifying historical reference files. NCBI table 5.
RSCU species means average accession-specific RSCU; gene summaries pool codons.
CODEML omega values are imported from the archived M0 summary, not newly estimated.
"""
from pathlib import Path
from collections import Counter, defaultdict
import argparse, json, math, platform
import numpy as np
import pandas as pd
from scipy import stats
from Bio import SeqIO, __version__ as biopython_version
from Bio.Data import CodonTable

GENES = 'atp6 atp8 cox1 cox2 cox3 cytb nad1 nad2 nad3 nad4 nad4l nad5 nad6'.split()
CODE = CodonTable.unambiguous_dna_by_id[5].forward_table
FAMILIES = defaultdict(list)
for codon, aa in CODE.items(): FAMILIES[aa].append(codon)

def canon(acc):
    return acc.rsplit('.', 1)[0] if acc.rsplit('.', 1)[-1].isdigit() else acc

def gene_name(feature):
    name = feature.qualifiers.get('gene', [''])[0].lower().replace('-', '')
    if not name:
        product=feature.qualifiers.get('product',[''])[0].lower()
        if 'nadh' in product: name='nad'+product.split()[-1]
        elif 'oxidase' in product: name={'i':'cox1','ii':'cox2','iii':'cox3','1':'cox1','2':'cox2','3'
:'cox3'}.get(product.split()[-1],'')
        elif 'atp' in product: name='atp'+product.split()[-1]
        elif 'cytochrome b' in product: name='cytb'
    aliases = {'cob': 'cytb', 'cytb': 'cytb', 'coi': 'cox1', 'coii': 'cox2', 'coiii': 'cox3'}
    name = aliases.get(name, name)
    if name in ['co1','co2','co3']: name='cox'+name[-1]
    return 'nad'+name[2:] if name.startswith('nd') else name

def rscu(counts):
    values = {}
    for aa, codons in FAMILIES.items():
        n = sum(counts[c] for c in codons)
        values.update({c: counts[c] * len(codons) / n if n else float('nan') for c in codons})
    return values

def composition(sequence):
    p = [sequence[i::3] for i in range(3)]
    gc = [100 * (v.count('G') + v.count('C')) / len(v) for v in p]
    a,t,g,c = (p[2].count(b) for b in 'ATGC')
    return dict(GC1=gc[0], GC2=gc[1], GC3=gc[2], GC12=(gc[0]+gc[1])/2,
                PR2_A3_A3T3=a/(a+t), PR2_G3_G3C3=g/(g+c))

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--root', type=Path, default=Path(__file__).resolve().parents[1])
    parser.add_argument('--out', type=Path, default=None)
    args=parser.parse_args(); root=args.root.resolve(); out=args.out or root/'results'
    out.mkdir(parents=True, exist_ok=True)
    ref=root/'data/reference'; raw=root/'data/genbank'
    manifest=pd.read_csv(root/'data/accessions_30.tsv', sep='\t')
    assert len(manifest)==30 and manifest.accession.nunique()==30
    source={canon(f.stem):f for f in raw.glob('*.gb')}
    records=[]; long=[]; qc=[]; comps=[]; genome=[]; gene_rows=[]; gene_counts=defaultdict(Counter)
    total=0
    for row in manifest.itertuples():
        acc=str(row.accession); species=str(row.species)
        species='Helicoverpa armigera' if species.startswith('Helicoverpa armigera') else species
        rec=SeqIO.read(source[canon(acc)], 'genbank'); pooled=Counter(); seqparts=[]; genes_seen=[]
        for feature in rec.features:
            if feature.type!='CDS': continue
            gene=gene_name(feature)
```

```
            if gene not in GENES: raise ValueError(f'Unrecognized CDS: {acc} {gene}')
            offset=int(feature.qualifiers.get('codon_start',['1'])[0])-1
            seq=str(feature.extract(rec.seq)).upper()[offset:]
            valid=[seq[i:i+3] for i in range(0,len(seq)-2,3) if seq[i:i+3] in CODE]
            counts=Counter(valid); pooled.update(counts); seqparts.extend(valid);
genes_seen.append(gene)
            # Third-position synonyms are evaluated within the first two bases.
            syn=[c for c in valid if any(CODE.get(c[:2]+b)==CODE[c] for b in 'ACGT' if b!=c[2])]
            gc3s=100*sum(c[2] in 'GC' for c in syn)/len(syn)

gene_rows.append(dict(accession=acc,gene=gene,codons=len(valid),codon_start=offset+1,GC3s=gc3s))
            gene_counts[gene].update(counts)
        assert sorted(genes_seen)==sorted(GENES), (acc,genes_seen)
        sequence=''.join(seqparts); records.append((acc,sequence)); total+=len(seqparts)
        qc.append(dict(accession=acc,species=species,CDS=len(genes_seen),sense_codons=len(seqparts)))
        comps.append(dict(accession=acc,species=species,**composition(sequence)))
        for c,value in sorted(rscu(pooled).items()):

long.append(dict(accession=acc,species=species,amino_acid=CODE[c],codon=c,count=pooled[c],RSCU=value))
        count=Counter(str(rec.seq).upper()); n=sum(count[b] for b in 'ACGT')
        genome.append(dict(accession=acc,species=species,length=len(rec),unambiguous_bases=n,
                           **{b:100*count[b]/n for b in 'ACGT'},
                           AT_skew=(count['A']-count['T'])/(count['A']+count['T']),
                           GC_skew=(count['G']-count['C'])/(count['G']+count['C'])))
    assert total==111739 and len(gene_rows)==390
    df=pd.DataFrame(long); df.to_csv(out/'rscu_30.tsv',sep='\t',index=False)
    means=df.groupby(['species','amino_acid','codon'],as_index=False).agg(genomes=('accession','size')
,mean_RSCU=('RSCU','mean'))
    means.to_csv(out/'rscu_medias_14_especies.tsv',sep='\t',index=False)
    assert means.species.nunique()==14 and len(means)==14*62
    pd.DataFrame(genome).to_csv(out/'genome_composition.tsv',sep='\t',index=False)
    pd.DataFrame(qc).to_csv(out/'codon_qc_30.tsv',sep='\t',index=False)
    comp=pd.DataFrame(comps);
comp.to_csv(out/'Codon_PR2_Neutrality_30mitogenomes.tsv',sep='\t',index=False)
    species=comp.groupby('species',as_index=False)[['GC3','GC12','PR2_A3_A3T3','PR2_G3_G3C3']].mean()
    species.to_csv(out/'species_composition.tsv',sep='\t',index=False)
    x=species.GC3.to_numpy();y=species.GC12.to_numpy();fit=stats.linregress(x,y)
    residuals=y-(fit.intercept+fit.slope*x); crit=stats.t.ppf(.975,len(x)-2)
    diagnostics=dict(n=len(x),slope=fit.slope,intercept=fit.intercept,r_squared=fit.rvalue**2,
                     p_value=fit.pvalue,slope_ci95=[fit.slope-
crit*fit.stderr,fit.slope+crit*fit.stderr],
                     shapiro_p=stats.shapiro(residuals).pvalue,
                     abs_residual_GC3_pearson_r=stats.pearsonr(abs(residuals),x).statistic,
                     abs_residual_GC3_pearson_p=stats.pearsonr(abs(residuals),x).pvalue)
    (out/'neutrality_diagnostics.json').write_text(json.dumps(diagnostics,indent=2))
    loo=[]
    for i,name in enumerate(species.species):
        f=stats.linregress(np.delete(x,i),np.delete(y,i))
        loo.append(dict(omitted_species=name,slope=f.slope,intercept=f.intercept,r_squared=f.rvalue**2
,p_value=f.pvalue))
    pd.DataFrame(loo).to_csv(out/'neutrality_leave_one_out.tsv',sep='\t',index=False)
    # Read archived estimates; do not treat this as a CODEML rerun.
    omega=pd.read_csv(ref/'dnds_M0_30/dNdS_M0_13genes_final30.csv').set_index('Gene').omega

gdf=pd.DataFrame(gene_rows);gdf.to_csv(out/'gene_accession_codon_metrics.tsv',sep='\t',index=False)
    summary=[]
    for gene in GENES:
        sub=gdf[gdf.gene==gene];rv=rscu(gene_counts[gene]);group=next((g for g in ['cox','nad','atp']
if gene.startswith(g)),'cytb')
        summary.append(dict(gene=gene,group=group,CDS=len(sub),sense_codons=int(sub.codons.sum()),
                            mean_GC3s=sub.GC3s.mean(),mean_RSCU_AT_end=float(np.nanmean([v for c,v in
rv.items() if c[-1] in 'AT'])),omega=float(omega[gene])))
    gs=pd.DataFrame(summary);gs.to_csv(out/'Supplementary_S1.tsv',sep='\t',index=False)
    groups=gs.groupby('group',as_index=False).agg(genes=('gene','size'),CDS=('CDS','sum'),sense_codons
=('sense_codons','sum'),
          mean_GC3s=('mean_GC3s','mean'),mean_RSCU_AT_end=('mean_RSCU_AT_end','mean'),mean_omega=('ome
ga','mean'))
    groups.to_csv(out/'Supplementary_S2.tsv',sep='\t',index=False)
    corr=[]
    for variable in ['mean_GC3s','mean_RSCU_AT_end']:
        pear=stats.pearsonr(gs[variable],gs.omega);spear=stats.spearmanr(gs[variable],gs.omega)
        corr.append(dict(variable=variable,n=13,Pearson_r=pear.statistic,Pearson_p=pear.pvalue,Spearma
n_rho=spear.statistic,Spearman_p=spear.pvalue))
    pd.DataFrame(corr).to_csv(out/'Supplementary_S3.tsv',sep='\t',index=False)
    # Identity-based audit uses full nucleotide sequences, no species-name guessing.
    allrec=[(f,SeqIO.read(f,'genbank')) for f in sorted(raw.glob('*.gb'))]
    chosen={canon(a) for a in manifest.accession}|{'MN793322'};duplicates=[]
    for f,rec in allrec:
        if canon(f.stem) in chosen:continue
        matches=[(ff,rr) for ff,rr in allrec if canon(ff.stem) in chosen and
```

```
str(rr.seq).upper()==str(rec.seq).upper()]
        for ff,rr in matches:duplicates.append(dict(excluded=f.stem,retained=ff.stem,length=len(rec),s
pecies=rec.annotations.get('organism','')))
    pd.DataFrame(duplicates).to_csv(out/'Supplementary_S4.tsv',sep='\t',index=False)
    # Compare reconstructed data with archived outputs using normalized accession versions.
    old=pd.read_csv(ref/'rscu_30.tsv',sep='\t');old['key']=old.accession.map(canon);df['key']=df.acces
sion.map(canon)
    match=df.merge(old,on=['key','codon'],suffixes=('_new','_old'),validate='one_to_one')
    assert len(match)==30*62
    maxdiff=float(abs(match.RSCU_new-match.RSCU_old).max());assert maxdiff<1e-6
    oldmeans=pd.read_csv(ref/'rscu_medias_14_especies.tsv',sep='\t')
    m=means.merge(oldmeans,on=['species','codon'],suffixes=('_new','_old'),validate='one_to_one')
    assert len(m)==14*62 and abs(m.mean_RSCU_new-m.mean_RSCU_old).max()<1e-6
    with (out/'codonw_input_30.fasta').open('w') as f:
        for acc,seq in records:f.write(f'>{acc}\n{seq}\n')
    historical={canon(r.id):str(r.seq) for r in SeqIO.parse(ref/'codonw_input_30.fasta','fasta')}
    assert all(historical[canon(a)]==s for a,s in records)
    checks=dict(CDS_records=len(gene_rows),sense_codons=total,RSCU_records=len(match),RSCU_max_abs_dif
ference=maxdiff,
                species_RSCU_records=len(m),CodonW_input_matches=True,duplicate_pairs=len(duplicates),
                codon_start_values=sorted(set(gdf.codon_start)),Python=platform.python_version(),Biopy
thon=biopython_version,
                external_phylogeny_and_CODEML_rerun=False)
    (out/'validation.json').write_text(json.dumps(checks,indent=2))
    print(json.dumps(checks,indent=2));print(gs.to_string(index=False));print(diagnostics)
if __name__=='__main__': main()
```

02_prepare_phylogeny.py

Repository path scripts/02_prepare_phylogeny.py

```
#!/usr/bin/env python3
"""Reconstruct final alignment from archived cleaned gene alignments.
This does not reconstruct the earlier MACSE alignment/cleaning operation.
"""
from pathlib import Path
import json
import pandas as pd
from Bio import SeqIO
from Bio.Seq import Seq
from Bio.SeqRecord import SeqRecord
GENES = 'atp6 atp8 cox1 cox2 cox3 cytb nad1 nad2 nad3 nad4 nad4l nad5 nad6'.split()
def canon(acc):
    return acc.rsplit('.',1)[0] if acc.rsplit('.',1)[-1].isdigit() else acc
root=Path(__file__).resolve().parents[1]
out=root/'results/phylogeny';out.mkdir(parents=True,exist_ok=True)
manifest=pd.read_csv(root/'data/accessions_30.tsv',sep='\t')
selected={canon(a) for a in manifest.accession}|{'MN793322'}
parts={};ids=None;start=1;partition=['#nexus','begin sets;']
for gene in GENES:
    records={r.id:str(r.seq).upper() for r in
SeqIO.parse(root/f'data/clean_gene_alignments/{gene}_clean_NT.fasta','fasta') if canon(r.id) in
selected}
    assert len(records)==31 and len({len(s) for s in records.values()})==1
    length=len(next(iter(records.values())))
    assert length%3==0 and all('!' not in s for s in records.values())
    if ids is None:ids=set(records)
    assert set(records)==ids
    parts[gene]=records
    partition.append(f'charset {gene} = {start}-{start+length-1};');start+=length
alignment=[SeqRecord(Seq(''.join(parts[g][a] for g in GENES)),id=a,description='') for a in
sorted(ids)]
assert start-1==11229
SeqIO.write(alignment,out/'PCG13_31_aligned.fasta','fasta')
partition.append('end;');(out/'PCG13_31_partitions.nex').write_text('\n'.join(partition)+'\n')
old={r.id:str(r.seq) for r in
SeqIO.parse(root/'data/reference/phylogeny_31/PCG13_31_aligned.fasta','fasta')}
assert {r.id:str(r.seq) for r in alignment}==old
# Derive a fresh 2-million-generation file from the archived initial MrBayes file.
# Only ngen, append flag and output filename are changed; models/partitions remain.
import re
text=(root/'data/reference/phylogeny_31/PCG13_31_BI.nex').read_text()
text=re.sub(r'ngen=\d+','ngen=2000000',text)
text=text.replace('append=yes','append=no').replace('filename=PCG13_31_BI','filename=BI_new')
(out/'BI_fresh.nex').write_text(text)
(out/'alignment_validation.json').write_text(json.dumps(dict(sequences=31,positions=11229,archived_ali
gnment_identical=True),indent=2))
print('Verified 31 sequences x 11229 positions; fresh BI configuration prepared.')
```

03_plot_results.py

Repository path scripts/03_plot_results.py

```
#!/usr/bin/env python3
"""Reconstructed plotting code with portable paths.
Replots tables using the documented analysis choices; visual styling may differ
from historical figures. ENC values and omega are imported archived estimates.
"""
from pathlib import Path
from io import StringIO
import re
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import linkage,leaves_list
from scipy.spatial.distance import pdist
from scipy.stats import linregress
from Bio import Phylo
root=Path(__file__).resolve().parents[1];res=root/'results';out=res/'figures';out.mkdir(exist_ok=True)
def save(fig,name):
    fig.tight_layout()
    for ext in ['pdf','png']:fig.savefig(out/f'{name}.{ext}',dpi=300,bbox_inches='tight')
    plt.close(fig)
# Euclidean distance and Ward linkage; unscaled species mean RSCU.
d=pd.read_csv(res/'rscu_medias_14_especies.tsv',sep='\t')
m=d.pivot(index='species',columns='codon',values='mean_RSCU')
ri=leaves_list(linkage(pdist(m.to_numpy()),method='ward'))
ci=leaves_list(linkage(pdist(m.to_numpy().T),method='ward'))
m=m.iloc[ri,ci];fig,ax=plt.subplots(figsize=(18,7))
im=ax.imshow(m.to_numpy(),aspect='auto',cmap='YlOrRd',vmin=0,vmax=6)
ax.set_yticks(range(len(m)),m.index,fontstyle='italic',fontsize=9)
ax.set_xticks(range(len(m.columns)),m.columns,rotation=90,fontsize=7)
ax.set_xlabel('Codon');fig.colorbar(im,ax=ax,label='Mean RSCU');save(fig,'Figure2_RSCU')
# CodonW outputs remain archived inputs until an external rerun is completed.
e=pd.read_csv(root/'data/reference/Supplementary_ENC_GC3s_30.tsv',sep='\t')
e['species']=e.species.str.replace(r'^Helicoverpa armigera.*','Helicoverpa armigera',regex=True)
e=e.groupby('species',as_index=False)[['ENC','GC3s']].mean()
fig,ax=plt.subplots(figsize=(8,6));x=np.linspace(.001,.999,500)
ax.plot(x,2+x+29/(x*x+(1-x)**2),color='grey',label='Expected ENC')
for i,r in e.iterrows():ax.scatter(r.GC3s,r.ENC,color=plt.cm.tab20(i),label=r.species)
ax.set(xlabel='GC3s
(fraction)',ylabel='ENC');ax.legend(bbox_to_anchor=(1.02,1),fontsize=7);save(fig,'Figure3_ENC')
s=pd.read_csv(res/'species_composition.tsv',sep='\t')
fig,ax=plt.subplots(figsize=(8,6))
for i,r in s.iterrows():ax.scatter(r.PR2_G3_G3C3,r.PR2_A3_A3T3,color=plt.cm.tab20(i),label=r.species)
ax.axhline(.5,color='grey',ls='--');ax.axvline(.5,color='grey',ls='--')
ax.set(xlim=(0,1),ylim=(0,1),xlabel='G3 / (G3 + C3)',ylabel='A3 / (A3 + T3)')
ax.legend(bbox_to_anchor=(1.02,1),fontsize=7);save(fig,'Figure4_PR2')
f=linregress(s.GC3,s.GC12);fig,ax=plt.subplots(figsize=(8,6))
for i,r in s.iterrows():ax.scatter(r.GC3,r.GC12,color=plt.cm.tab20(i),label=r.species)
x=np.linspace(s.GC3.min(),s.GC3.max(),100);ax.plot(x,f.intercept+f.slope*x)
ax.set(xlabel='GC3 (%)',ylabel='GC12 (%)');ax.legend(bbox_to_anchor=(1.02,1),fontsize=7)
ax.text(.03,.97,f'R squared = {f.rvalue**2:.4f}; p =
{f.pvalue:.5f}',transform=ax.transAxes,va='top');save(fig,'Figure5_Neutrality')
g=pd.read_csv(res/'Supplementary_S1.tsv',sep='\t');fig,ax=plt.subplots(figsize=(9,5))
ax.bar(g.gene,g.omega,color='#365C70');ax.set(xlabel='Gene',ylabel='dN/dS (archived CODEML
M0)');save(fig,'Figure6_M0')
# Trees are re-rendered as cladograms with original support labels retained.
for label,path,fmt in [('ML','PCG13_31_ML.treefile','newick'),('BI','PCG13_31_BI.con.tre','nexus')]:
    text=(root/f'data/reference/phylogeny_31/{path}').read_text()
    if fmt=='nexus':
        text=re.sub(r'(\))\[&prob=([^,\]]+)[^\]]*\]', lambda m:m[1]+m[2], text)
        text=re.sub(r'\[[^\]]*\]','',text)
    tree=Phylo.read(StringIO(text),fmt)
    outgroup=next(t for t in tree.get_terminals() if 'MN793322' in t.name)
    tree.root_with_outgroup(outgroup)
    for clade in tree.find_clades():clade.branch_length=1
    fig,ax=plt.subplots(figsize=(11,12));Phylo.draw(tree,axes=ax,do_show=False)
    ax.set_xlabel('Cladogram');save(fig,f'Figure1_{label}_cladogram')
print('Figures written to',out)
```

04_run_external.py

Repository path scripts/04_run_external.py

```
#!/usr/bin/env python3
"""Run optional external analyses in new output directories.
Executables must already be installed on PATH. These expensive analyses were
not rerun during package preparation. Run 02_prepare_phylogeny.py first for ML/BI.
"""
import argparse, shutil, subprocess
from pathlib import Path
root=Path(__file__).resolve().parents[1]
p=argparse.ArgumentParser(description=__doc__)
p.add_argument('analysis',choices=['ml','bi','m0','codonw']);args=p.parse_args()
if args.analysis in ['ml','bi']:
    wd=root/'results/phylogeny'
    if not (wd/'PCG13_31_aligned.fasta').exists():p.error('Run 02_prepare_phylogeny.py first.')
    if args.analysis=='ml':
        if (wd/'ML_new.log').exists():p.error('ML_new already exists; use a fresh working copy.')
        cmd=['iqtree2','-s','PCG13_31_aligned.fasta','-p','PCG13_31_partitions.nex','-
m','MFP+MERGE','-B','1000','--alrt','1000','-T','AUTO','-o','MN793322.1','--seed','20260927','--
prefix','ML_new']
    else:
        if list(wd.glob('BI_new*')):p.error('BI_new already exists; use a fresh working copy.')
        cmd=['mb','BI_fresh.nex']
    subprocess.run(cmd,cwd=wd,check=True)
elif args.analysis=='m0':
    dst=root/'results/M0_new'
    if dst.exists():p.error('M0_new already exists; use a fresh working copy.')
    shutil.copytree(root/'data/reference/dnds_M0_30',dst)
    for ctl in sorted(dst.glob('*/codeml.ctl')):
        subprocess.run(['codeml','codeml.ctl'],cwd=ctl.parent,check=True)
else:
    wd=root/'results'
    if not (wd/'codonw_input_30.fasta').exists():p.error('Run 01_reproduce_tables.py first.')
    if (wd/'codonw_new.tsv').exists():p.error('codonw_new.tsv already exists.')
    subprocess.run(['codonw','codonw_input_30.fasta','codonw_new.tsv','codonw_new.blk','-code','4','-
enc','-gc3s','-machine','-nomenu','-silent','-noblk'],cwd=wd,check=True)
```

Recovered historical Python blocks

The following blocks were recovered verbatim from complete Python heredocs in the provided terminal export after removal of the history display prefix. Their syntax was checked. They were not executed as a sequence: some are diagnostics, intermediate attempts or patch operations. The source index gives original working directories and line ranges. The incomplete opening RSCU fragment is not represented as a complete executable script. Historical absolute paths are retained only in this provenance appendix.

block 01 source 0028 0062

```
from pathlib import Path
from Bio import SeqIO
from Bio.Data import CodonTable
base = Path.cwd()
source = base / "01_raw_data/final_genbank"
out = base / "revision_30_noctuidos"
duplicates = {
    "NC_022689.1", "NC_022185.1", "NC_023118.1",
    "NC_053623", "NC_061705.1", "NC_054179",
    "NC_061562", "NC_015835.1", "NC_019622"
}
sense = set(CodonTable.unambiguous_dna_by_id[5].forward_table)
records = []
for path in sorted(source.glob("*.gb")):
    accession = path.stem
    if accession in duplicates or accession == "MN793322.1":
        continue
    record = SeqIO.read(path, "genbank")
    codons = []
    for feature in record.features:
        if feature.type != "CDS":
            continue
        sequence = str(feature.extract(record.seq)).upper()
        codons.extend(sequence[i:i+3]
                      for i in range(0, len(sequence) - 2, 3)
                      if sequence[i:i+3] in sense)
    records.append((accession, "".join(codons)))
assert len(records) == 30
```

```
assert sum(len(sequence) // 3 for _, sequence in records) == 111739
with (out / "codonw_input_30.fasta").open("w") as file:
    for accession, sequence in records:
        file.write(f">{accession}\n")
        for i in range(0, len(sequence), 80):
            file.write(sequence[i:i+80] + "\n")
print("Entrada CodonW:", len(records), "mitogenomas")
```

block 02 source 0069 0138

```
from pathlib import Path
import csv
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
folder = Path.cwd()
enc = pd.read_csv(folder / "enc_gc3s_30.tsv", sep=r"\s+")
qc = pd.read_csv(folder / "codon_qc_30.tsv", sep="\t")
enc = enc.rename(columns={"title": "accession", "Nc": "ENC"})
data = qc[["accession", "species"]].merge(
    enc[["accession", "ENC", "GC3s"]],
    on="accession", validate="one_to_one"
)
if len(data) != 30 or data[["ENC", "GC3s"]].isna().any().any():
    raise ValueError("La tabla no contiene 30 valores completos")
if not data["GC3s"].between(0, 1).all():
    raise ValueError("GC3s está fuera del intervalo 0–1")
data["genus"] = data["species"].str.split().str[0]
data = data.sort_values(["species", "accession"])
data[["accession", "species", "ENC", "GC3s"]].to_csv(
    folder / "Supplementary_ENC_GC3s_30.tsv",
    sep="\t", index=False, float_format="%.6f"
)
colors = {
    "Agrotis": "#1f77b4",
    "Athetis": "#e69f00",
    "Helicoverpa": "#009e73",
    "Mamestra": "#d55e00",
    "Mythimna": "#56b4e9",
    "Sesamia": "#795548",
    "Spodoptera": "#c62828",
    "Trichoplusia": "#424242",
}
x = np.linspace(0, 1, 500)
expected = 2 + x + 29 / (x*x + (1-x)*(1-x))
fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharey=False)
for ax in axes:
    ax.plot(x, expected, color="black", linewidth=1.5,
            label="Expected ENC")
    for genus, group in data.groupby("genus"):
        ax.scatter(group["GC3s"], group["ENC"], s=55,
                   color=colors[genus], edgecolor="white",
                   linewidth=0.5, label=genus, zorder=3)
    ax.set_xlabel("GC3s")
    ax.set_ylabel("Effective number of codons (ENC)")
    ax.grid(alpha=0.2)
axes[0].set_title("Full expected curve")
axes[0].set_xlim(0, 1)
axes[0].set_ylim(20, 62)
pad_x = max(0.015, (data.GC3s.max() - data.GC3s.min()) * 0.2)
axes[1].set_title("Observed mitogenomes")
axes[1].set_xlim(max(0, data.GC3s.min()-pad_x),
                 data.GC3s.max()+pad_x)
axes[1].set_ylim(data.ENC.min()-2, data.ENC.max()+2)
handles, labels = axes[1].get_legend_handles_labels()
fig.legend(handles, labels, loc="lower center", ncol=5,
           frameon=False, bbox_to_anchor=(0.5, -0.04))
fig.tight_layout(rect=(0, 0.09, 1, 1))
for extension in ("png", "pdf"):
    fig.savefig(folder / f"Figure3_ENC_GC3s_30.{extension}",
                dpi=300, bbox_inches="tight")
plt.close(fig)
print("Mitogenomas:", len(data))
print("ENC:", round(data.ENC.min(), 2), "a",
      round(data.ENC.max(), 2))
print("GC3s:", round(data.GC3s.min(), 3), "a",
```

```
      round(data.GC3s.max(), 3))
print("Figura:", folder / "Figure3_ENC_GC3s_30.png")
```

block 03 source 0143 0196

```
from pathlib import Path
import pandas as pd
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
folder = Path.cwd()
data = pd.read_csv(folder / "Supplementary_ENC_GC3s_30.tsv", sep="\t")
data["species"] = data["species"].where(
    ~data["species"].str.startswith("Helicoverpa armigera"),
    "Helicoverpa armigera"
)
means = data.groupby("species", as_index=False)[["ENC", "GC3s"]].mean()
if len(means) != 14:
    raise ValueError(f"Se esperaban 14 especies; hay {len(means)}")
means.to_csv(folder / "Figure3_species_means_14.tsv",
             sep="\t", index=False, float_format="%.6f")
colors = {
    "Agrotis": "#1f77b4",
    "Athetis": "#e69f00",
    "Helicoverpa": "#009e73",
    "Mamestra": "#d55e00",
    "Mythimna": "#56b4e9",
    "Sesamia": "#795548",
    "Spodoptera": "#c62828",
    "Trichoplusia": "#424242",
}
markers = ["o", "s", "^", "D", "P", "X", "v", "<",
           ">", "h", "8", "p", "*", "H"]
fig, ax = plt.subplots(figsize=(9, 6))
x = np.linspace(0.035, 0.09, 400)
expected = 2 + x + 29 / (x*x + (1-x)*(1-x))
ax.plot(x, expected, color="black", linestyle="--", linewidth=1.8,
        label="Expected ENC curve")
for i, row in means.iterrows():
    genus = row["species"].split()[0]
    ax.scatter(row["GC3s"], row["ENC"], s=85,
               color=colors[genus], marker=markers[i],
               edgecolor="black", linewidth=0.5,
               label=row["species"], zorder=3)
ax.set_xlim(0.035, 0.09)
ax.set_ylim(28, 38)
ax.set_xlabel("GC3s")
ax.set_ylabel("Effective number of codons (ENC)")
ax.grid(alpha=0.2)
ax.legend(loc="center left", bbox_to_anchor=(1.02, 0.5),
          fontsize=8, frameon=False)
fig.tight_layout()
for extension in ("png", "pdf"):
    fig.savefig(folder / f"Figure3_ENC_GC3s_14_species_v2.{extension}",
                dpi=300, bbox_inches="tight")
plt.close(fig)
print("Especies:", len(means))
print("Figura:", folder / "Figure3_ENC_GC3s_14_species_v2.png")
```

block 04 source 0202 0215

```
import pandas as pd
from scipy.stats import linregress, pearsonr, spearmanr
for archivo, etiqueta in [
    ("Supplementary_ENC_GC3s_30.tsv", "30 mitogenomas"),
    ("Figure3_species_means_14.tsv", "14 medias por especie"),
]:
    df = pd.read_csv(archivo, sep="\t")
    print(f"\n{etiqueta}: {archivo}")
    print("Columnas:", ", ".join(df.columns))
    x, y = df["GC3s"], df["ENC"]
    reg = linregress(x, y)
    print(f"n={len(df)}; pendiente={reg.slope:.4f}; intercepto={reg.intercept:.4f};
R²={reg.rvalue**2:.4f}; p_regresión={reg.pvalue:.4g}")
    print(f"Pearson: r={pearsonr(x, y).statistic:.4f}; p={pearsonr(x, y).pvalue:.4g}")
    print(f"Spearman: rho={spearmanr(x, y).statistic:.4f}; p={spearmanr(x, y).pvalue:.4g}")
```

block 05 source 0219 0252

```
from pathlib import Path
import shutil
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import linregress
folder = Path.cwd()
df = pd.read_csv(folder / "Figure3_species_means_14.tsv", sep="\t")
reg = linregress(df["GC3s"], df["ENC"])
x = np.linspace(0.035, 0.090, 300)
enc_esperado = 2 + x + 29 / (x**2 + (1-x)**2)
fig, ax = plt.subplots(figsize=(9, 6))
ax.plot(x, enc_esperado, "--", color="#777777",
        linewidth=2, label="Expected ENC (GC3s only)")
ax.plot(x, reg.intercept + reg.slope*x, color="#15616D",
        linewidth=2, label=f"Observed regression (R² = {reg.rvalue**2:.3f})")
colores = plt.cm.tab20(np.linspace(0, 1, len(df)))
for (_, row), color in zip(df.iterrows(), colores):
    ax.scatter(row["GC3s"], row["ENC"], s=65, color=color,
               edgecolor="black", linewidth=0.4, label=row["species"], zorder=3)
ax.set(xlim=(0.035, 0.090), ylim=(28, 38),
       xlabel="GC3s", ylabel="Effective number of codons (ENC)")
ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left", fontsize=8,
          frameon=False)
ax.grid(alpha=0.2)
fig.tight_layout()
destino = Path("/mnt/e/Mitogenoma/nuevas figuras")
destino.mkdir(parents=True, exist_ok=True)
for extension in ("png", "pdf"):
    archivo = folder / f"Figure3_ENC_GC3s_regression_14_species.{extension}"
    fig.savefig(archivo, dpi=300, bbox_inches="tight")
    shutil.copy2(archivo, destino / archivo.name)
    print(destino / archivo.name)
plt.close(fig)
```

block 06 source 0263 0330

```
from pathlib import Path
import shutil
import pandas as pd
import matplotlib.pyplot as plt
from Bio import SeqIO
carpeta = Path.cwd()
tax = pd.read_csv("Supplementary_ENC_GC3s_30.tsv", sep="\t")
especie = dict(zip(tax["accession"].astype(str), tax["species"]))
filas = []
for rec in SeqIO.parse("codonw_input_30.fasta", "fasta"):
    acc = rec.id
    if acc not in especie:
        raise ValueError(f"Accesión no encontrada en la tabla: {acc}")
    seq = str(rec.seq).upper()
    if len(seq) % 3:
        raise ValueError(f"Secuencia fuera de marco: {acc}")
    p1, p2, p3 = seq[0::3], seq[1::3], seq[2::3]
    if set(seq) - set("ATGC"):
        raise ValueError(f"Bases ambiguas en: {acc}")
    a, t, g, c = (p3.count(base) for base in "ATGC")
    filas.append({
        "accession": acc,
        "species": especie[acc],
        "codons": len(p3),
        "GC1": 100*(p1.count("G")+p1.count("C"))/len(p1),
        "GC2": 100*(p2.count("G")+p2.count("C"))/len(p2),
        "GC3": 100*(g+c)/len(p3),
        "GC12": 50*((p1.count("G")+p1.count("C"))/len(p1)
                    +(p2.count("G")+p2.count("C"))/len(p2)),
        "PR2_A3_A3T3": a/(a+t),
        "PR2_G3_G3C3": g/(g+c)
    })
df = pd.DataFrame(filas)
assert len(df) == 30, f"Se esperaban 30 mitogenomas; se encontraron {len(df)}"
assert df["species"].nunique() == 14, "El número de especies no es 14"
df.to_csv("Codon_PR2_Neutrality_30mitogenomes.tsv", sep="\t", index=False)
medias = df.groupby("species", as_index=False).agg(
    n=("accession", "size"),
```

```
    PR2_AT=("PR2_A3_A3T3", "mean"),
    PR2_GC=("PR2_G3_G3C3", "mean")
)
medias.to_csv("Figure4_PR2_14_species.tsv", sep="\t", index=False)
fig, ax = plt.subplots(figsize=(8, 6))
for i, row in medias.iterrows():
    ax.scatter(row["PR2_GC"], row["PR2_AT"], s=70,
               color=plt.cm.tab20(i), edgecolor="black", linewidth=0.4,
               label=row["species"])
ax.axhline(0.5, color="gray", linestyle="--", linewidth=1)
ax.axvline(0.5, color="gray", linestyle="--", linewidth=1)
ax.set(xlim=(0, 1), ylim=(0, 1),
       xlabel="G3 / (G3 + C3)", ylabel="A3 / (A3 + T3)")
ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left",
          fontsize=8, frameon=False)
fig.tight_layout()
destino = Path("/mnt/e/Mitogenoma/nuevas figuras")
destino.mkdir(parents=True, exist_ok=True)
for extension in ("png", "pdf"):
    archivo = carpeta / f"Figure4_PR2_14_species.{extension}"
    fig.savefig(archivo, dpi=300, bbox_inches="tight")
    shutil.copy2(archivo, destino / archivo.name)
    print("Copiado:", destino / archivo.name)
plt.close(fig)
print("Mitogenomas:", len(df), "| Especies:", len(medias),
      "| Codones:", df["codons"].sum())
print("Rango PR2 A/T:", round(df["PR2_A3_A3T3"].min(), 3),
      "a", round(df["PR2_A3_A3T3"].max(), 3))
print("Rango PR2 G/C:", round(df["PR2_G3_G3C3"].min(), 3),
      "a", round(df["PR2_G3_G3C3"].max(), 3))
```

block 07 source 0333 0335

```
import pandas as pd
d = pd.read_csv("Supplementary_ENC_GC3s_30.tsv", sep="\t")
print(d.groupby("species").size().to_string())
```

block 08 source 0339 0408

```
from pathlib import Path
import shutil
import pandas as pd
import matplotlib.pyplot as plt
from Bio import SeqIO
carpeta = Path(__file__).resolve().parent
tax = pd.read_csv(carpeta / "Supplementary_ENC_GC3s_30.tsv", sep="\t")
tax["species"] = tax["species"].apply(
    lambda s: "Helicoverpa armigera"
    if str(s).startswith("Helicoverpa armigera") else s
)
especie = dict(zip(tax["accession"].astype(str), tax["species"]))
filas = []
for rec in SeqIO.parse(carpeta / "codonw_input_30.fasta", "fasta"):
    acc = rec.id
    if acc not in especie:
        raise ValueError(f"Accesión sin especie: {acc}")
    seq = str(rec.seq).upper()
    if len(seq) % 3 or set(seq) - set("ATGC"):
        raise ValueError(f"Secuencia inválida: {acc}")
    p1, p2, p3 = seq[0::3], seq[1::3], seq[2::3]
    a, t, g, c = (p3.count(b) for b in "ATGC")
    filas.append({
        "accession": acc, "species": especie[acc], "codons": len(p3),
        "GC1": 100*(p1.count("G")+p1.count("C"))/len(p1),
        "GC2": 100*(p2.count("G")+p2.count("C"))/len(p2),
        "GC3": 100*(g+c)/len(p3),
        "GC12": 50*((p1.count("G")+p1.count("C"))/len(p1)
                    +(p2.count("G")+p2.count("C"))/len(p2)),
        "PR2_A3_A3T3": a/(a+t),
        "PR2_G3_G3C3": g/(g+c)
    })
df = pd.DataFrame(filas)
assert len(df) == 30, f"Mitogenomas encontrados: {len(df)}"
assert df["species"].nunique() == 14, f"Especies encontradas: {df['species'].nunique()}"
df.to_csv(carpeta / "Codon_PR2_Neutrality_30mitogenomes.tsv",
          sep="\t", index=False)
medias = df.groupby("species", as_index=False).agg(
```

```
    n=("accession", "size"),
    PR2_AT=("PR2_A3_A3T3", "mean"),
    PR2_GC=("PR2_G3_G3C3", "mean")
)
medias.to_csv(carpeta / "Figure4_PR2_14_species.tsv",
              sep="\t", index=False)
fig, ax = plt.subplots(figsize=(8, 6))
for i, row in medias.iterrows():
    ax.scatter(row["PR2_GC"], row["PR2_AT"], s=70,
               color=plt.cm.tab20(i), edgecolor="black",
               linewidth=0.4, label=row["species"])
ax.axhline(0.5, color="gray", linestyle="--", linewidth=1)
ax.axvline(0.5, color="gray", linestyle="--", linewidth=1)
ax.set(xlim=(0, 1), ylim=(0, 1),
       xlabel="G3 / (G3 + C3)", ylabel="A3 / (A3 + T3)")
ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left",
          fontsize=8, frameon=False)
fig.tight_layout()
destino = Path("/mnt/e/Mitogenoma/nuevas figuras")
destino.mkdir(parents=True, exist_ok=True)
for ext in ("png", "pdf"):
    archivo = carpeta / f"Figure4_PR2_14_species.{ext}"
    fig.savefig(archivo, dpi=300, bbox_inches="tight")
    shutil.copy2(archivo, destino / archivo.name)
    print("Copiado:", destino / archivo.name)
plt.close(fig)
print("Mitogenomas:", len(df), "| Especies:", len(medias),
      "| Codones:", df["codons"].sum())
print("PR2 A/T:", round(df["PR2_A3_A3T3"].min(), 3),
      "a", round(df["PR2_A3_A3T3"].max(), 3))
print("PR2 G/C:", round(df["PR2_G3_G3C3"].min(), 3),
      "a", round(df["PR2_G3_G3C3"].max(), 3))
```

block 09 source 0415 0425

```
import pandas as pd
from scipy.stats import linregress
d = pd.read_csv("Codon_PR2_Neutrality_30mitogenomes.tsv", sep="\t")
m = d.groupby("species", as_index=False)[["GC3", "GC12"]].mean()
r = linregress(m["GC3"], m["GC12"])
m.to_csv("Figure5_neutrality_14_species.tsv", sep="\t", index=False)
print(f"Especies: {len(m)}")
print(f"Pendiente: {r.slope:.4f}")
print(f"Intercepto: {r.intercept:.4f}")
print(f"R²: {r.rvalue**2:.4f}")
print(f"p: {r.pvalue:.4g}")
```

block 10 source 0429 0463

```
from pathlib import Path
import shutil
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import linregress
carpeta = Path.cwd()
d = pd.read_csv("Figure5_neutrality_14_species.tsv", sep="\t")
assert len(d) == 14
r = linregress(d["GC3"], d["GC12"])
fig, ax = plt.subplots(figsize=(8, 6))
for i, row in d.iterrows():
    ax.scatter(row["GC3"], row["GC12"], s=75,
               color=plt.cm.tab20(i), edgecolor="black",
               linewidth=0.4, label=row["species"], zorder=3)
x = np.linspace(d["GC3"].min(), d["GC3"].max(), 100)
ax.plot(x, r.intercept + r.slope*x, color="#15616D", linewidth=2)
ax.set_xlabel("GC3 (%)")
ax.set_ylabel("GC12 (%)")
ax.text(0.03, 0.97,
        f"GC12 = {r.intercept:.3f} + {r.slope:.3f} × GC3\n"
        f"R² = {r.rvalue**2:.3f}; p = {r.pvalue:.4f}; n = {len(d)}",
        transform=ax.transAxes, va="top", fontsize=10)
ax.grid(alpha=0.2)
ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left",
          fontsize=8, frameon=False)
fig.tight_layout()
destino = Path("/mnt/e/Mitogenoma/nuevas figuras")
```

```
destino.mkdir(parents=True, exist_ok=True)
for ext in ("png", "pdf"):
    archivo = carpeta / f"Figure5_neutrality_14_species.{ext}"
    fig.savefig(archivo, dpi=300, bbox_inches="tight")
    shutil.copy2(archivo, destino / archivo.name)
    print("Copiado:", destino / archivo.name)
plt.close(fig)
```

block 11 source 0470 0488

```
from pathlib import Path
import pandas as pd
from Bio import SeqIO, Phylo
base = Path.cwd()
tabla = pd.read_csv(base / "revision_30_noctuidos/Supplementary_ENC_GC3s_30.tsv", sep="\t")
elegidas = set(tabla["accession"].astype(str))
arbol = base / "04_alignments/concatenated_final40/PCG13_final40_by_gene.treefile"
puntas = {x.name for x in Phylo.read(arbol, "newick").get_terminals()}
print("Accesiones elegidas:", len(elegidas))
print("Presentes en árbol:", len(elegidas & puntas))
print("Faltan en árbol:", sorted(elegidas - puntas))
genes = "atp6 atp8 cox1 cox2 cox3 cytb nad1 nad2 nad3 nad4 nad4l nad5 nad6".split()
for gen in genes:
    ruta = base / f"04_alignments/gene_alignments_final40_clean/{gen}_clean_NT.fasta"
    secuencias = list(SeqIO.parse(ruta, "fasta"))
    ids = {s.id for s in secuencias}
    print(gen, "total:", len(secuencias),
          "elegidas:", len(ids & elegidas),
          "faltan:", sorted(elegidas - ids))
```

block 12 source 0492 0498

```
from Bio import SeqIO, Phylo
a = "04_alignments/gene_alignments_final40_clean/cox1_clean_NT.fasta"
t = "04_alignments/concatenated_final40/PCG13_final40_by_gene.treefile"
print("Ejemplos del alineamiento:")
print(*[r.id for r in list(SeqIO.parse(a, "fasta"))[:20]], sep="\n")
print("\nEjemplos del árbol:")
print(*[x.name for x in Phylo.read(t, "newick").get_terminals()[:20]], sep="\n")
```

block 13 source 0502 0541

```
from pathlib import Path
import pandas as pd
from Bio import SeqIO, Phylo
base = Path.cwd()
salida = base / "revision_30_noctuidos/dnds_input_30"
origen = base / "04_alignments/gene_alignments_final40_clean"
arbol_origen = base / "04_alignments/concatenated_final40/PCG13_final40_by_gene.treefile"
def sin_version(nombre):
    return nombre.rsplit(".", 1)[0] if nombre.rsplit(".", 1)[-1].isdigit() else nombre
tabla = pd.read_csv(base / "revision_30_noctuidos/Supplementary_ENC_GC3s_30.tsv",
                    sep="\t")
elegidas = {sin_version(str(x)) for x in tabla["accession"]}
assert len(elegidas) == 30
arbol = Phylo.read(arbol_origen, "newick")
puntas = list(arbol.get_terminals())
assert len({sin_version(x.name) for x in puntas}) == len(puntas)
assert elegidas <= {sin_version(x.name) for x in puntas}
genes = "atp6 atp8 cox1 cox2 cox3 cytb nad1 nad2 nad3 nad4 nad4l nad5 nad6".split()
seleccionados = {}
for gen in genes:
    registros = list(SeqIO.parse(origen / f"{gen}_clean_NT.fasta", "fasta"))
    ids = {sin_version(r.id) for r in registros}
    assert len(ids) == len(registros), f"Identificadores repetidos: {gen}"
    assert elegidas <= ids, f"Faltan accesiones en {gen}: {sorted(elegidas-ids)}"
    elegidos = [r for r in registros if sin_version(r.id) in elegidas]
    assert len(elegidos) == 30
    assert len({len(r.seq) for r in elegidos}) == 1
    assert len(elegidos[0].seq) % 3 == 0
    seleccionados[gen] = elegidos
salida.mkdir(parents=True, exist_ok=True)
for gen, registros in seleccionados.items():
    SeqIO.write(registros, salida / f"{gen}_30_NT.fasta", "fasta")
for punta in list(arbol.get_terminals()):
    if sin_version(punta.name) not in elegidas:
```

```
        arbol.prune(punta)
assert len(arbol.get_terminals()) == 30
Phylo.write(arbol, salida / "PCG13_pruned_30.nwk", "newick")
print("Preparados:", len(genes), "alineamientos de 30 secuencias")
print("Árbol:", len(arbol.get_terminals()), "puntas")
print("Carpeta:", salida)
```

block 14 source 0546 0569

```
from pathlib import Path
origen = Path("run_codeml_M0_final40.py")
destino = Path("revision_30_noctuidos/run_codeml_M0_30.py")
s = origen.read_text()
cambios = [
    ('project\n    / "04_alignments"\n    / "gene_alignments_final40_clean"',
     'project\n    / "revision_30_noctuidos"\n    / "dnds_input_30"'),
    ('project\n    / "04_alignments"\n    / "concatenated_final40"\n    /
"PCG13_final40_by_gene.treefile"',
     'project\n    / "revision_30_noctuidos"\n    / "dnds_input_30"\n    / "PCG13_pruned_30.nwk"'),
    ('project / "07_dnds_M0_final40"',
     'project / "revision_30_noctuidos" / "dnds_M0_30"'),
    ('f"{gene}_clean_NT.fasta"', 'f"{gene}_30_NT.fasta"'),
]
for viejo, nuevo in cambios:
    if s.count(viejo) != 1:
        raise SystemExit(f"No se encontró exactamente una vez: {viejo}")
    s = s.replace(viejo, nuevo)
s = s.replace("final40", "final30")
s = s.replace("40 mitogenomas", "30 mitogenomas")
s = s.replace("!= 40", "!= 30")
s = s.replace("no 40", "no 30")
s = s.replace('f"40 1\\n', 'f"30 1\\n')
destino.write_text(s)
print("Programa creado:", destino)
```

block 15 source 0574 0616

```
from pathlib import Path
import shutil
import pandas as pd
import matplotlib.pyplot as plt
base = Path.cwd()
d = pd.read_csv(base / "dnds_M0_30/dNdS_M0_13genes_final30.csv")
assert len(d) == 13 and d["status"].eq("OK").all()
grupos = {
    "atp6": "ATP synthase", "atp8": "ATP synthase",
    "cox1": "Cytochrome c oxidase", "cox2": "Cytochrome c oxidase",
    "cox3": "Cytochrome c oxidase", "cytb": "Cytochrome b",
}
d["group"] = d["Gene"].map(lambda g: grupos.get(g, "NADH dehydrogenase"))
colores = {
    "ATP synthase": "#D08B3E",
    "Cytochrome c oxidase": "#247BA0",
    "Cytochrome b": "#54A24B",
    "NADH dehydrogenase": "#B65F73",
}
fig, ax = plt.subplots(figsize=(9, 5))
barras = ax.bar(d["Gene"], d["omega"],
                color=d["group"].map(colores), edgecolor="black",
                linewidth=0.4)
ax.bar_label(barras, fmt="%.4f", padding=3, fontsize=8, rotation=90)
ax.set_ylabel("dN/dS (ω), CODEML M0")
ax.set_xlabel("Mitochondrial protein-coding gene")
ax.set_ylim(0, d["omega"].max()*1.35)
ax.spines[["top", "right"]].set_visible(False)
for nombre, color in colores.items():
    ax.bar([], [], color=color, label=nombre)
ax.legend(frameon=False, ncol=2, fontsize=8, loc="upper left")
fig.tight_layout()
destino = Path("/mnt/e/Mitogenoma/nuevas figuras")
destino.mkdir(parents=True, exist_ok=True)
for ext in ("png", "pdf"):
    archivo = base / f"Figure6_dNdS_M0_30.{ext}"
    fig.savefig(archivo, dpi=300, bbox_inches="tight")
    shutil.copy2(archivo, destino / archivo.name)
    print("Copiado:", destino / archivo.name)
tabla = destino / "dNdS_M0_13genes_30.csv"
```

```
shutil.copy2(base / "dnds_M0_30/dNdS_M0_13genes_final30.csv", tabla)
print("Copiado:", tabla)
plt.close(fig)
```

block 16 source 0625 0635

```
from pathlib import Path
from Bio import SeqIO
for p in Path(".").rglob("*"):
    if p.suffix.lower() not in {".fasta", ".fa", ".fas", ".fna"}:
        continue
    try:
        registros = list(SeqIO.parse(p, "fasta"))
    except Exception:
        continue
    if len(registros) == 31 and len({len(r.seq) for r in registros}) == 1:
        print(p.resolve(), "longitud:", len(registros[0].seq))
```

block 17 source 0639 0649

```
from Bio import SeqIO
import pandas as pd
ruta = "04_alignments/concatenated_final40/PCG13_concatenated_40mitogenomes.fasta"
r = list(SeqIO.parse(ruta, "fasta"))
print("Concatenado original:", len(r), "secuencias")
print("Longitudes:", sorted({len(x.seq) for x in r}))
print("Mapa taxonómico:")
m = pd.read_csv("06_codon_usage_final40/accession_species_final40.csv")
print(m[m.astype(str).apply(
    lambda c: c.str.contains("Ostrinia", case=False, na=False)
).any(axis=1)].to_string(index=False))
```

block 18 source 0654 0665

```
from Bio import SeqIO
from pathlib import Path
carpeta = Path("04_alignments/gene_alignments_final40_clean")
genes = "atp6 atp8 cox1 cox2 cox3 cytb nad1 nad2 nad3 nad4 nad4l nad5 nad6".split()
total = 0
for gen in genes:
    r = list(SeqIO.parse(carpeta / f"{gen}_clean_NT.fasta", "fasta"))
    largo = len(r[0].seq)
    total += largo
    externo = any(x.id == "MN793322.1" for x in r)
    print(gen, largo, "bp; Ostrinia:", externo)
print("Longitud concatenada:", total, "bp")
```

block 19 source 0669 0713

```
from pathlib import Path
from Bio import SeqIO
from Bio.Seq import Seq
from Bio.SeqRecord import SeqRecord
import pandas as pd
base = Path.cwd()
origen = base / "04_alignments/gene_alignments_final40_clean"
salida = base / "revision_30_noctuidos/phylogeny_31"
salida.mkdir(parents=True, exist_ok=True)
tabla = pd.read_csv(base / "revision_30_noctuidos/Supplementary_ENC_GC3s_30.tsv",
                    sep="\t")
def canon(x):
    return x.rsplit(".", 1)[0] if x.rsplit(".", 1)[-1].isdigit() else x
elegidas = {canon(str(x)) for x in tabla["accession"]}
elegidas.add(canon("MN793322.1"))
assert len(elegidas) == 31
genes = "atp6 atp8 cox1 cox2 cox3 cytb nad1 nad2 nad3 nad4 nad4l nad5 nad6".split()
partes = {}
nombres = None
for gen in genes:
    registros = list(SeqIO.parse(origen / f"{gen}_clean_NT.fasta", "fasta"))
    seleccion = {r.id: str(r.seq).upper() for r in registros
                 if canon(r.id) in elegidas}
    assert len(seleccion) == 31, f"{gen}: {len(seleccion)} secuencias"
    assert len({len(s) for s in seleccion.values()}) == 1
    assert len(next(iter(seleccion.values()))) % 3 == 0
```

```
    assert all("!" not in s for s in seleccion.values()), f"Frameshift: {gen}"
    if nombres is None:
        nombres = set(seleccion)
    assert set(seleccion) == nombres, f"Nombres distintos en {gen}"
    partes[gen] = seleccion
orden = sorted(nombres)
secuencias = [
    SeqRecord(Seq("".join(partes[gen][acc] for gen in genes)),
              id=acc, description="")
    for acc in orden
]
archivo = salida / "PCG13_31_aligned.fasta"
SeqIO.write(secuencias, archivo, "fasta")
assert len(secuencias) == 31
assert {len(r.seq) for r in secuencias} == {11229}
print("Alineamiento:", archivo)
print("Secuencias:", len(secuencias))
print("Longitud:", len(secuencias[0].seq), "bp")
print("Grupo externo:", "MN793322.1" in orden)
```

block 20 source 0717 0731

```
from pathlib import Path
from Bio import SeqIO
origen = Path("~/noctuidae_mitogenomes_v2/04_alignments/gene_alignments_final40_clean").expanduser()
genes = "atp6 atp8 cox1 cox2 cox3 cytb nad1 nad2 nad3 nad4 nad4l nad5 nad6".split()
inicio = 1
lineas = ["#nexus", "begin sets;"]
for gen in genes:
    largo = len(next(SeqIO.parse(origen / f"{gen}_clean_NT.fasta", "fasta")).seq)
    fin = inicio + largo - 1
    lineas.append(f"    charset {gen} = {inicio}-{fin};")
    inicio = fin + 1
assert inicio - 1 == 11229
lineas.append("end;")
Path("PCG13_31_partitions.nex").write_text("\n".join(lineas) + "\n")
print("Particiones:", len(genes), "| Longitud total:", inicio - 1)
```

block 21 source 0736 0743

```
from Bio import SeqIO, Phylo
aln = list(SeqIO.parse("PCG13_31_aligned.fasta", "fasta"))
tips = [x.name for x in Phylo.read("PCG13_31_ML.treefile", "newick").get_terminals()]
print("Alineamiento:", len(aln), "secuencias;", len(aln[0].seq), "bp")
print("Árbol ML:", len(tips), "puntas")
print("Identificadores coinciden:", {x.id for x in aln} == set(tips))
print("\nEsquema seleccionado:")
print(open("PCG13_31_ML.best_scheme.nex").read()[:2500])
```

block 22 source 0747 0782

```
from pathlib import Path
import shutil
import pandas as pd
import matplotlib.pyplot as plt
from Bio import Phylo
datos = pd.read_csv("../Supplementary_ENC_GC3s_30.tsv", sep="\t")
def canon(s):
    return str(s).split(".")[0]
nombres = dict(zip(datos.accession.map(canon), datos.species))
nombres["MN793322"] = "Ostrinia nubilalis"
arbol = Phylo.read("PCG13_31_ML.treefile", "newick")
assert len(arbol.get_terminals()) == 31
def etiqueta(c):
    if not c.is_terminal():
        return None
    especie = nombres[canon(c.name)].replace(" ", r"\ ")
    return rf"$\it{{{especie}}}$  {c.name}"
def apoyo(c):
    if c.is_terminal():
        return None
    return c.name or (f"{c.confidence:g}" if c.confidence is not None else None)
fig, ax = plt.subplots(figsize=(15, 12))
Phylo.draw(arbol, axes=ax, do_show=False,
           label_func=etiqueta, branch_labels=apoyo)
ax.set_xlabel("Substitutions per site")
```

```
ax.set_ylabel("")
ax.set_yticks([])
fig.tight_layout()
destino = Path("/mnt/e/Mitogenoma/nuevas figuras")
destino.mkdir(parents=True, exist_ok=True)
for ext in ("png", "pdf"):
    archivo = Path(f"Figure1_phylogeny_ML_31.{ext}")
    fig.savefig(archivo, dpi=300, bbox_inches="tight")
    shutil.copy2(archivo, destino / archivo.name)
    print("Copiado:", destino / archivo.name)
plt.close(fig)
```

block 23 source 0785 0791

```
from Bio import Phylo
t = Phylo.read("PCG13_31_ML.treefile", "newick")
internos = t.get_nonterminals()
print("Nodos con más de dos descendientes:",
      [(len(c.clades), c.name) for c in internos if len(c.clades) > 2])
print("Ramas internas de longitud cero o casi cero:",
      sum((c.branch_length or 0) < 1e-7 for c in internos))
```

block 24 source 0794 0828

```
from pathlib import Path
import shutil
import pandas as pd
import matplotlib.pyplot as plt
from Bio import Phylo
d = pd.read_csv("../Supplementary_ENC_GC3s_30.tsv", sep="\t")
nombres = dict(zip(d.accession.astype(str).str.split(".").str[0], d.species))
nombres["MN793322"] = "Ostrinia nubilalis"
t = Phylo.read("PCG13_31_ML.treefile", "newick")
t.root_with_outgroup("MN793322.1")
print("Descendientes de la raíz:", len(t.root.clades))
print("Politomías internas:", sum(len(c.clades) > 2 for c in t.get_nonterminals()))
assert all(len(c.clades) <= 2 for c in t.get_nonterminals())
for c in t.find_clades():
    if c is not t.root:
        c.branch_length = 1
def etiqueta(c):
    if not c.is_terminal():
        return None
    especie = nombres[c.name.split(".")[0]].replace(" ", r"\ ")
    return rf"$\it{{{especie}}}$  {c.name}"
fig, ax = plt.subplots(figsize=(15, 13))
Phylo.draw(t, axes=ax, do_show=False, label_func=etiqueta,
           branch_labels=lambda c: c.name if not c.is_terminal() else None)
ax.set_xlabel("Topology shown as a cladogram; branch lengths are not to scale")
ax.set_ylabel("")
ax.set_yticks([])
fig.tight_layout()
destino = Path("/mnt/e/Mitogenoma/nuevas figuras")
for ext in ("png", "pdf"):
    archivo = Path(f"Figure1_phylogeny_ML_31_cladogram.{ext}")
    fig.savefig(archivo, dpi=300, bbox_inches="tight")
    shutil.copy2(archivo, destino / archivo.name)
    print("Copiado:", destino / archivo.name)
plt.close(fig)
```

block 25 source 0832 0870

```
from Bio import SeqIO
from pathlib import Path
r = list(SeqIO.parse("PCG13_31_aligned.fasta", "fasta"))
assert len(r) == 31 and {len(x.seq) for x in r} == {11229}
assert len({x.id for x in r}) == 31
assert any(x.id == "MN793322.1" for x in r)
lineas = [
    "#NEXUS",
    "begin data;",
    "  dimensions ntax=31 nchar=11229;",
    "  format datatype=dna missing=? gap=-;",
    "  matrix",
]
lineas += [f"  {x.id}  {str(x.seq).upper()}" for x in r]
```

```
lineas += [
    "  ;",
    "end;",
    "begin mrbayes;",
    "  set autoclose=yes nowarn=yes;",
    "  outgroup MN793322.1;",
    "  charset P1 = 1-678 679-843 5941-6951 6952-7305 10693-11229;",
    "  charset P2 = 844-2373 2374-3054 3055-3843 3844-4995;",
    "  charset P3 = 4996-5940 7306-8643 8644-8940 8941-10692;",
    "  partition genes = 3: P1, P2, P3;",
    "  set partition=genes;",
    "  lset applyto=(all) nst=6 rates=invgamma;",
    "  unlink revmat=(all) statefreq=(all) shape=(all) pinvar=(all);",
    "  prset applyto=(all) ratepr=variable;",
    "  mcmcp ngen=1000000 samplefreq=1000 printfreq=10000",
    "        diagnfreq=10000 nruns=2 nchains=4",
    "        filename=PCG13_31_BI;",
    "  mcmc;",
    "  sump;",
    "  sumt;",
    "end;",
]
archivo = Path("PCG13_31_BI.nex")
archivo.write_text("\n".join(lineas) + "\n")
print("Archivo MrBayes:", archivo, "| 31 secuencias | 11229 bp | 3 particiones")
```

block 26 source 0891 0898

```
from pathlib import Path
original = Path("PCG13_31_BI.nex")
texto = original.read_text()
anterior = "mcmcp ngen=1000000"
assert texto.count(anterior) == 1, "No se encontró una única línea mcmcp esperada"
texto = texto.replace(anterior, "mcmcp ngen=1000000 append=yes", 1)
Path("PCG13_31_BI_continuar.nex").write_text(texto)
print("Preparado: PCG13_31_BI_continuar.nex")
```

block 27 source 0906 0914

```
from pathlib import Path
filas = [x.split() for x in Path("PCG13_31_BI.mcmc").read_text().splitlines()
         if x.strip() and x.lstrip()[0].isdigit()]
print("Generación y AvgStdDev:", filas[-1][0], filas[-1][-1])
datos = [x.split() for x in Path("PCG13_31_BI.pstat").read_text().splitlines()[2:]
         if len(x.split()) >= 9]
print("ESS mínimo:", min(float(x[6]) for x in datos))
print("PSRF mínimo y máximo:", min(float(x[8]) for x in datos),
      max(float(x[8]) for x in datos))
```

Original CODEML settings

These original gene specific control files are included verbatim. PAML icode 4 denotes the invertebrate mitochondrial code in that program; its numbering differs from NCBI table numbering. All gene specific alignments, trees and reference outputs accompany the package.

atp6 CODEML control

```
seqfile = atp6.phy
     treefile = tree_final30.nwk
      outfile = atp6_M0.mlc

        noisy = 3
      verbose = 1
      runmode = 0

      seqtype = 1
    CodonFreq = 2
        clock = 0
       aaDist = 0

        model = 0
      NSsites = 0

        icode = 4
```

```
    fix_kappa = 0
        kappa = 2

    fix_omega = 0
        omega = 0.1

    fix_alpha = 1
        alpha = 0
       Malpha = 0
        ncatG = 8

        getSE = 0
 RateAncestor = 0

   Small_Diff = 0.5e-6

    cleandata = 1
       method = 0
```

atp8 CODEML control

```
seqfile = atp8.phy
     treefile = tree_final30.nwk
      outfile = atp8_M0.mlc

        noisy = 3
      verbose = 1
      runmode = 0

      seqtype = 1
    CodonFreq = 2
        clock = 0
       aaDist = 0

        model = 0
      NSsites = 0

        icode = 4
    fix_kappa = 0
        kappa = 2

    fix_omega = 0
        omega = 0.1

    fix_alpha = 1
        alpha = 0
       Malpha = 0
        ncatG = 8

        getSE = 0
 RateAncestor = 0

   Small_Diff = 0.5e-6

    cleandata = 1
       method = 0
```

cox1 CODEML control

```
seqfile = cox1.phy
     treefile = tree_final30.nwk
      outfile = cox1_M0.mlc

        noisy = 3
      verbose = 1
      runmode = 0

      seqtype = 1
    CodonFreq = 2
        clock = 0
       aaDist = 0

        model = 0
      NSsites = 0

        icode = 4
    fix_kappa = 0
```

```
        kappa = 2

    fix_omega = 0
        omega = 0.1

    fix_alpha = 1
        alpha = 0
       Malpha = 0
        ncatG = 8

        getSE = 0
 RateAncestor = 0

   Small_Diff = 0.5e-6

    cleandata = 1
       method = 0
```

cox2 CODEML control

```
seqfile = cox2.phy
     treefile = tree_final30.nwk
      outfile = cox2_M0.mlc

        noisy = 3
      verbose = 1
      runmode = 0

      seqtype = 1
    CodonFreq = 2
        clock = 0
       aaDist = 0

        model = 0
      NSsites = 0

        icode = 4
    fix_kappa = 0
        kappa = 2

    fix_omega = 0
        omega = 0.1

    fix_alpha = 1
        alpha = 0
       Malpha = 0
        ncatG = 8

        getSE = 0
 RateAncestor = 0

   Small_Diff = 0.5e-6

    cleandata = 1
       method = 0
```

cox3 CODEML control

```
seqfile = cox3.phy
     treefile = tree_final30.nwk
      outfile = cox3_M0.mlc

        noisy = 3
      verbose = 1
      runmode = 0

      seqtype = 1
    CodonFreq = 2
        clock = 0
       aaDist = 0

        model = 0
      NSsites = 0

        icode = 4
    fix_kappa = 0
        kappa = 2
```

```
    fix_omega = 0
        omega = 0.1

    fix_alpha = 1
        alpha = 0
       Malpha = 0
        ncatG = 8

        getSE = 0
 RateAncestor = 0

   Small_Diff = 0.5e-6

    cleandata = 1
       method = 0
```

cytb CODEML control

```
seqfile = cytb.phy
     treefile = tree_final30.nwk
      outfile = cytb_M0.mlc

        noisy = 3
      verbose = 1
      runmode = 0

      seqtype = 1
    CodonFreq = 2
        clock = 0
       aaDist = 0

        model = 0
      NSsites = 0

        icode = 4
    fix_kappa = 0
        kappa = 2

    fix_omega = 0
        omega = 0.1

    fix_alpha = 1
        alpha = 0
       Malpha = 0
        ncatG = 8

        getSE = 0
 RateAncestor = 0

   Small_Diff = 0.5e-6

    cleandata = 1
       method = 0
```

nad1 CODEML control

```
seqfile = nad1.phy
     treefile = tree_final30.nwk
      outfile = nad1_M0.mlc

        noisy = 3
      verbose = 1
      runmode = 0

      seqtype = 1
    CodonFreq = 2
        clock = 0
       aaDist = 0

        model = 0
      NSsites = 0

        icode = 4
    fix_kappa = 0
        kappa = 2

    fix_omega = 0
```

```
        omega = 0.1

    fix_alpha = 1
        alpha = 0
       Malpha = 0
        ncatG = 8

        getSE = 0
 RateAncestor = 0

   Small_Diff = 0.5e-6

    cleandata = 1
       method = 0
```

nad2 CODEML control

```
seqfile = nad2.phy
     treefile = tree_final30.nwk
      outfile = nad2_M0.mlc

        noisy = 3
      verbose = 1
      runmode = 0

      seqtype = 1
    CodonFreq = 2
        clock = 0
       aaDist = 0

        model = 0
      NSsites = 0

        icode = 4
    fix_kappa = 0
        kappa = 2

    fix_omega = 0
        omega = 0.1

    fix_alpha = 1
        alpha = 0
       Malpha = 0
        ncatG = 8

        getSE = 0
 RateAncestor = 0

   Small_Diff = 0.5e-6

    cleandata = 1
       method = 0
```

nad3 CODEML control

```
seqfile = nad3.phy
     treefile = tree_final30.nwk
      outfile = nad3_M0.mlc

        noisy = 3
      verbose = 1
      runmode = 0

      seqtype = 1
    CodonFreq = 2
        clock = 0
       aaDist = 0

        model = 0
      NSsites = 0

        icode = 4
    fix_kappa = 0
        kappa = 2

    fix_omega = 0
        omega = 0.1
```

```
    fix_alpha = 1
        alpha = 0
       Malpha = 0
        ncatG = 8

        getSE = 0
 RateAncestor = 0

   Small_Diff = 0.5e-6

    cleandata = 1
       method = 0
```

nad4 CODEML control

```
seqfile = nad4.phy
     treefile = tree_final30.nwk
      outfile = nad4_M0.mlc

        noisy = 3
      verbose = 1
      runmode = 0

      seqtype = 1
    CodonFreq = 2
        clock = 0
       aaDist = 0

        model = 0
      NSsites = 0

        icode = 4
    fix_kappa = 0
        kappa = 2

    fix_omega = 0
        omega = 0.1

    fix_alpha = 1
        alpha = 0
       Malpha = 0
        ncatG = 8

        getSE = 0
 RateAncestor = 0

   Small_Diff = 0.5e-6

    cleandata = 1
       method = 0
```

nad4l CODEML control

```
seqfile = nad4l.phy
     treefile = tree_final30.nwk
      outfile = nad4l_M0.mlc

        noisy = 3
      verbose = 1
      runmode = 0

      seqtype = 1
    CodonFreq = 2
        clock = 0
       aaDist = 0

        model = 0
      NSsites = 0

        icode = 4
    fix_kappa = 0
        kappa = 2

    fix_omega = 0
        omega = 0.1

    fix_alpha = 1
```

```
        alpha = 0
       Malpha = 0
        ncatG = 8

        getSE = 0
 RateAncestor = 0

   Small_Diff = 0.5e-6

    cleandata = 1
       method = 0
```

nad5 CODEML control

```
seqfile = nad5.phy
     treefile = tree_final30.nwk
      outfile = nad5_M0.mlc

        noisy = 3
      verbose = 1
      runmode = 0

      seqtype = 1
    CodonFreq = 2
        clock = 0
       aaDist = 0

        model = 0
      NSsites = 0

        icode = 4
    fix_kappa = 0
        kappa = 2

    fix_omega = 0
        omega = 0.1

    fix_alpha = 1
        alpha = 0
       Malpha = 0
        ncatG = 8

        getSE = 0
 RateAncestor = 0

   Small_Diff = 0.5e-6

    cleandata = 1
       method = 0
```

nad6 CODEML control

```
seqfile = nad6.phy
     treefile = tree_final30.nwk
      outfile = nad6_M0.mlc

        noisy = 3
      verbose = 1
      runmode = 0

      seqtype = 1
    CodonFreq = 2
        clock = 0
       aaDist = 0

        model = 0
      NSsites = 0

        icode = 4
    fix_kappa = 0
        kappa = 2

    fix_omega = 0
        omega = 0.1

    fix_alpha = 1
        alpha = 0
```

```
       Malpha = 0
        ncatG = 8

        getSE = 0
 RateAncestor = 0

   Small_Diff = 0.5e-6

    cleandata = 1
       method = 0
```

Original phylogenetic partition definitions

```
#nexus
begin sets;
    charset atp6 = 1-678;
    charset atp8 = 679-843;
    charset cox1 = 844-2373;
    charset cox2 = 2374-3054;
    charset cox3 = 3055-3843;
    charset cytb = 3844-4995;
    charset nad1 = 4996-5940;
    charset nad2 = 5941-6951;
    charset nad3 = 6952-7305;
    charset nad4 = 7306-8643;
    charset nad4l = 8644-8940;
    charset nad5 = 8941-10692;
    charset nad6 = 10693-11229;
end;
```

Original MrBayes configuration

PCG13_31_BI.nex contains the 31 sequence nucleotide matrix as well as this analysis block. The matrix is supplied in the repository. The continuation configuration depends on the original run state; use the prepared BI_fresh.nex for a new run.

```
begin mrbayes;
  set autoclose=yes nowarn=yes;
  outgroup MN793322.1;
  charset P1 = 1-678 679-843 5941-6951 6952-7305 10693-11229;
  charset P2 = 844-2373 2374-3054 3055-3843 3844-4995;
  charset P3 = 4996-5940 7306-8643 8644-8940 8941-10692;
  partition genes = 3: P1, P2, P3;
  set partition=genes;
  lset applyto=(all) nst=6 rates=invgamma;
  unlink revmat=(all) statefreq=(all) shape=(all) pinvar=(all);
  prset applyto=(all) ratepr=variable;
  mcmcp ngen=1000000 samplefreq=1000 printfreq=10000
        diagnfreq=10000 nruns=2 nchains=4
        filename=PCG13_31_BI;
  mcmc;
  sump;
  sumt;
end;
```

PCG13_31_BI_continuar.nex contains the 31 sequence nucleotide matrix as well as this analysis block. The matrix is supplied in the repository. The continuation configuration depends on the original run state; use the prepared BI_fresh.nex for a new run.

```
begin mrbayes;
  set autoclose=yes nowarn=yes;
  outgroup MN793322.1;
  charset P1 = 1-678 679-843 5941-6951 6952-7305 10693-11229;
  charset P2 = 844-2373 2374-3054 3055-3843 3844-4995;
  charset P3 = 4996-5940 7306-8643 8644-8940 8941-10692;
  partition genes = 3: P1, P2, P3;
  set partition=genes;
  lset applyto=(all) nst=6 rates=invgamma;
  unlink revmat=(all) statefreq=(all) shape=(all) pinvar=(all);
  prset applyto=(all) ratepr=variable;
  mcmcp ngen=2000000 append=yes samplefreq=1000 printfreq=10000
        diagnfreq=10000 nruns=2 nchains=4
        filename=PCG13_31_BI;
  mcmc;
  sump;
```

```
  sumt;
end;
```

Source checksums

```
4ae2c35442784efbd881f4e0a295df6a4d6fa99ba2ac928a5a4153cbcddd784e  scripts/01_reproduce_tables.py
a881af0333a5f062da5d17ffe85f1e75b0e801f0987ff9acffa07a8a839e671b  scripts/02_prepare_phylogeny.py
7c64beed6919e62c0b7f5a610f22aaf25f6639c97caf8b347bd80f804ad6704a  scripts/03_plot_results.py
b1bf8ae01d7b5713edf4004a322ce3042fa296cf5cf8b8ce15fc5f71c3b92107  scripts/04_run_external.py
```

Appendix: CDS extraction and sense-codon filtering

Reconstructed reproducibility script, verified against the 30 GenBank records in the accompanying repository. This is not claimed to be recovered original historical source code.

Validation: 390 CDS (13 genes x 30 accessions); 111,739 sense codons; all 390 CDS have codon_start=1. The script excludes 286 stop triplets and 106 terminal bases; no ambiguous triplets were encountered. Excluded terminal bases are individual nucleotides, not complete codons.

Execution from repository root:

```
python scripts/05_extract_filter_CDS.py --genbank-dir data/genbank \
  --accessions data/accessions_30.tsv --out-dir results/cds_filtering
```

Source: scripts/05_extract_filter_CDS.py

```
#!/usr/bin/env python3
"""Reconstructed, auditable CDS extraction and sense-codon filtering.
Input: 30 accession IDs (one per line) and corresponding GenBank files.
Output: CDS-level QC and filtered sequences. Requires Biopython.
This script is a documented reconstruction, not claimed as historical code.
"""
import argparse
import csv
from pathlib import Path
from Bio import SeqIO
from Bio.Data import CodonTable
GENES = {'atp6','atp8','cox1','cox2','cox3','cytb','nad1','nad2',
         'nad3','nad4','nad4l','nad5','nad6'}
SENSE = set(CodonTable.unambiguous_dna_by_id[5].forward_table)
def gene_name(feature):
    name = feature.qualifiers.get('gene', [''])[0].lower().replace('-', '')
    if not name:
        product=feature.qualifiers.get('product',[''])[0].lower()
        if 'nadh' in product: name='nad'+product.split()[-1]
        elif 'oxidase' in product: name={'i':'cox1','ii':'cox2','iii':'cox3','1':'cox1','2':'cox2','3':'cox3'}.get(product.split()[-1],'')
        elif 'atp' in product: name='atp'+product.split()[-1]
        elif 'cytochrome b' in product: name='cytb'
    aliases = {'cob': 'cytb', 'cytb': 'cytb', 'coi': 'cox1', 'coii': 'cox2', 'coiii': 'cox3'}
    name = aliases.get(name, name)
    if name in ['co1','co2','co3']: name='cox'+name[-1]
    return 'nad'+name[2:] if name.startswith('nd') else name
def process_feature(feature, genome):
    # Biopython respects complement(), join(), and annotated strand.
    seq = str(feature.extract(genome.seq)).upper().replace('U', 'T')
    start = int(feature.qualifiers.get('codon_start', ['1'])[0])
    if start not in (1, 2, 3):
        raise ValueError(f'Invalid codon_start: {start}')
    seq = seq[start-1:]
    triplets = [seq[i:i+3] for i in range(0, len(seq)//3*3, 3)]
    valid = [c for c in triplets if c in SENSE]
    return start, valid, sum(c in CodonTable.unambiguous_dna_by_id[5].stop_codons for c in triplets), sum(
        len(c) == 3 and c not in SENSE and c not in CodonTable.unambiguous_dna_by_id[5].stop_codons for c in triplets), len(seq)%3
def main():
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument('--genbank-dir', type=Path, required=True)
    p.add_argument('--accessions', type=Path, required=True,
                   help='TSV with accession column, or plain text one accession per line')
    p.add_argument('--out-dir', type=Path, default=Path('results/cds_filtering'))
    args = p.parse_args()
    lines = [s.strip() for s in args.accessions.read_text().splitlines() if s.strip()]
    accessions = [s.split('\t')[0] for s in lines]
    if accessions[0].lower() == 'accession':
        accessions.pop(0)
    if len(accessions) != 30 or len(set(accessions)) != 30:
        raise ValueError('Expected 30 unique accessions')
    args.out_dir.mkdir(parents=True, exist_ok=True)
    rows, sequences, n_codons = [], [], 0
    for accession in accessions:
        candidates = list(args.genbank_dir.glob(accession + '.*'))
        if len(candidates) != 1:
            raise FileNotFoundError(f'{accession}: expected one GenBank file; found {len(candidates)}')
        record = SeqIO.read(candidates[0], 'genbank')
        found = []
        for f in record.features:
            if f.type != 'CDS':
                continue
            gene = gene_name(f)
            if gene not in GENES:
                raise ValueError(f'Unexpected CDS gene {accession}: {gene}')
            start, valid, stops, ambiguous, remainder = process_feature(f, record)
            # Stop detection uses table 5, not the standard nuclear code.
            rows.append(dict(accession=accession, gene=gene, strand=f.location.strand,
                             codon_start=start, sense_codons=len(valid),
```

```
                            excluded_stop=stops, excluded_ambiguous=ambiguous,
                            excluded_terminal_bases=remainder))
                sequences.append((f'{accession}|{gene}', ''.join(valid)))
                n_codons += len(valid)
                found.append(gene)
            if len(found) != 13 or set(found) != GENES or len(set(found)) != 13:
                raise ValueError(f'{accession}: missing/duplicate genes: {found}')
        if len(rows) != 390 or n_codons != 111739:
            raise ValueError(f'QC mismatch: {len(rows)} CDS, {n_codons} sense codons')
        if any(r['codon_start'] != 1 for r in rows):
            raise ValueError('Expected codon_start=1 for all 390 CDS')
        with (args.out_dir/'cds_filtering_qc.tsv').open('w', newline='') as h:
            w = csv.DictWriter(h, fieldnames=list(rows[0]), delimiter='\t')
            w.writeheader(); w.writerows(rows)
        with (args.out_dir/'filtered_CDS_390.fasta').open('w') as h:
            for name, seq in sequences:
                h.write(f'>{name}\n')
                for i in range(0, len(seq), 80):
                    h.write(seq[i:i+80]+'\n')
        print(f'Validated {len(rows)} CDS; {n_codons} sense codons; table 5')
if __name__ == '__main__':
    main()
```